

Code Quality with SonarQube

Mastering SonarQube

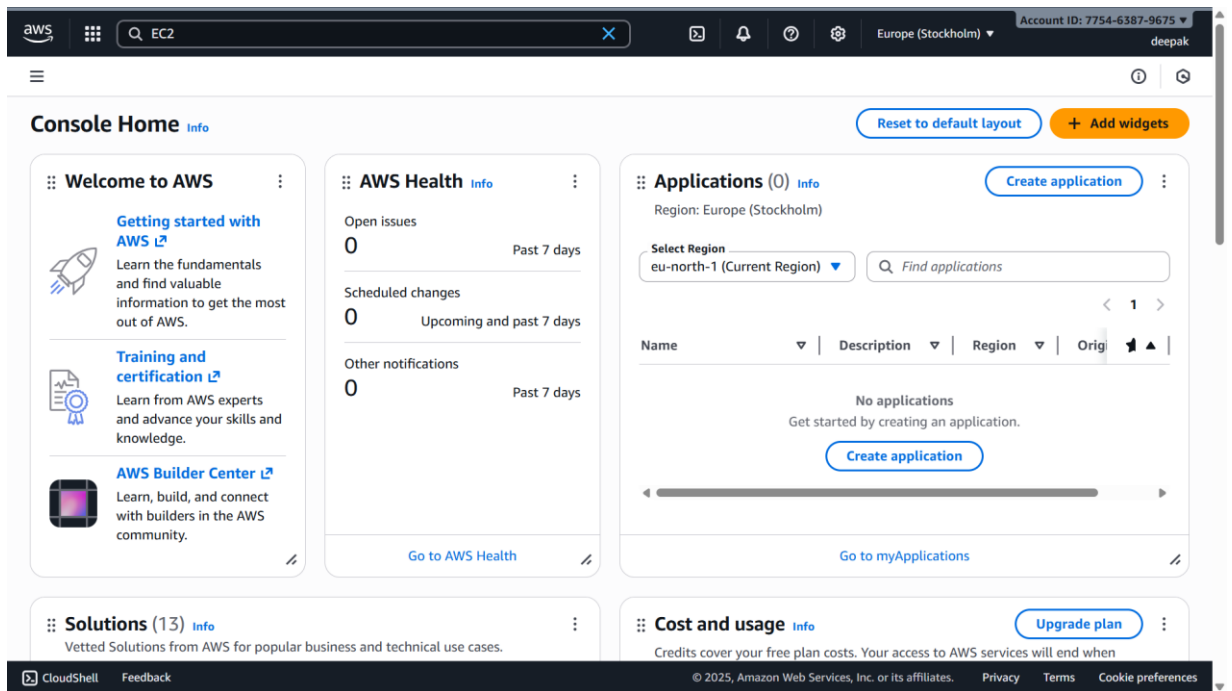
Table of Contents

Activity 1: SonarQube installation on AWS Platform	3
Activity 2: Creating Project in SonarQube	13
Activity 3: Static Code Analysis using SonarQube.....	16
Activity 4: Static Code Analysis using Sonar Scanner	19
Activity 5: Static Code Analysis on spring boot application cloned from Gitlab.....	21
Activity 6: Integrating SonarQube and Gitlab with Jenkins to build pipeline	23

Activity 1: SonarQube installation on AWS Platform

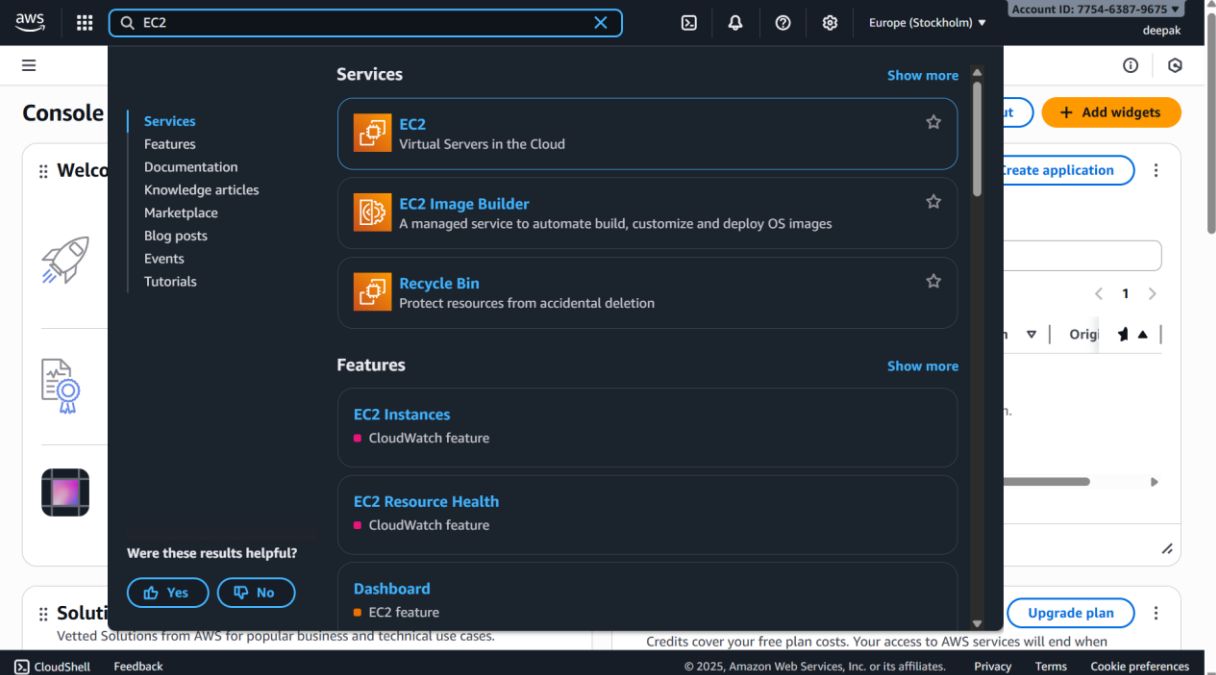
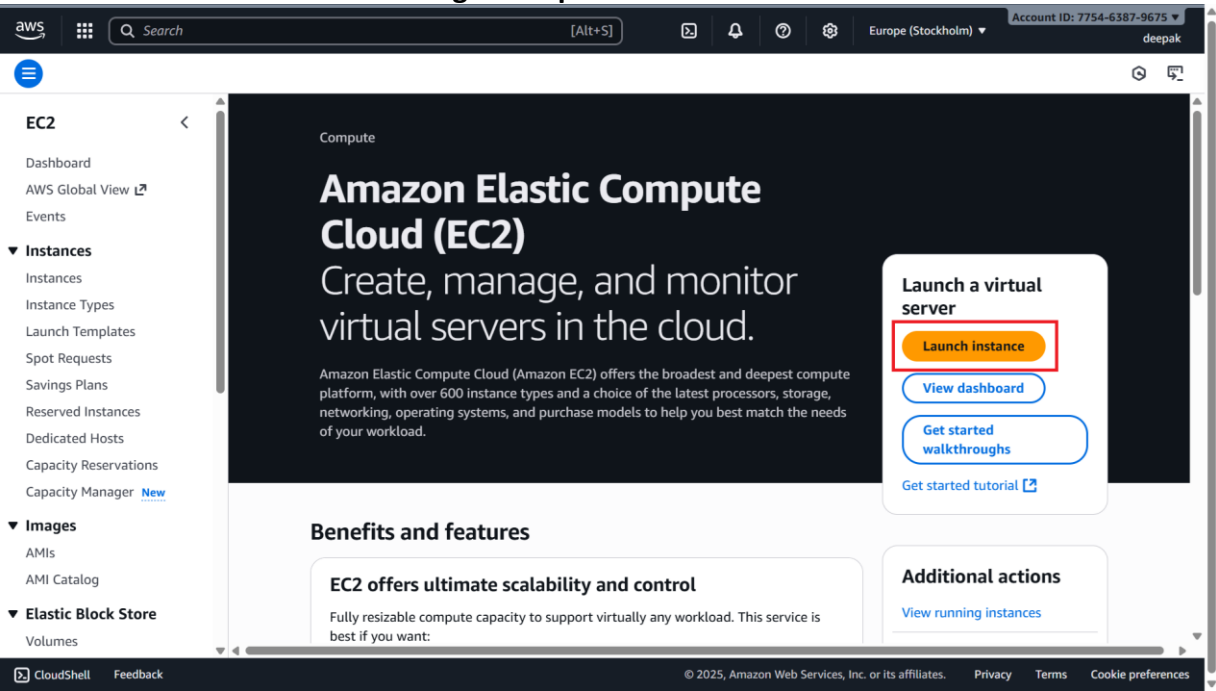
1. Log in to AWS Console

- Visit <https://aws.amazon.com/console>
- Sign in with your credentials



2. Navigate to EC2 Dashboard

- In the AWS Console, search for EC2

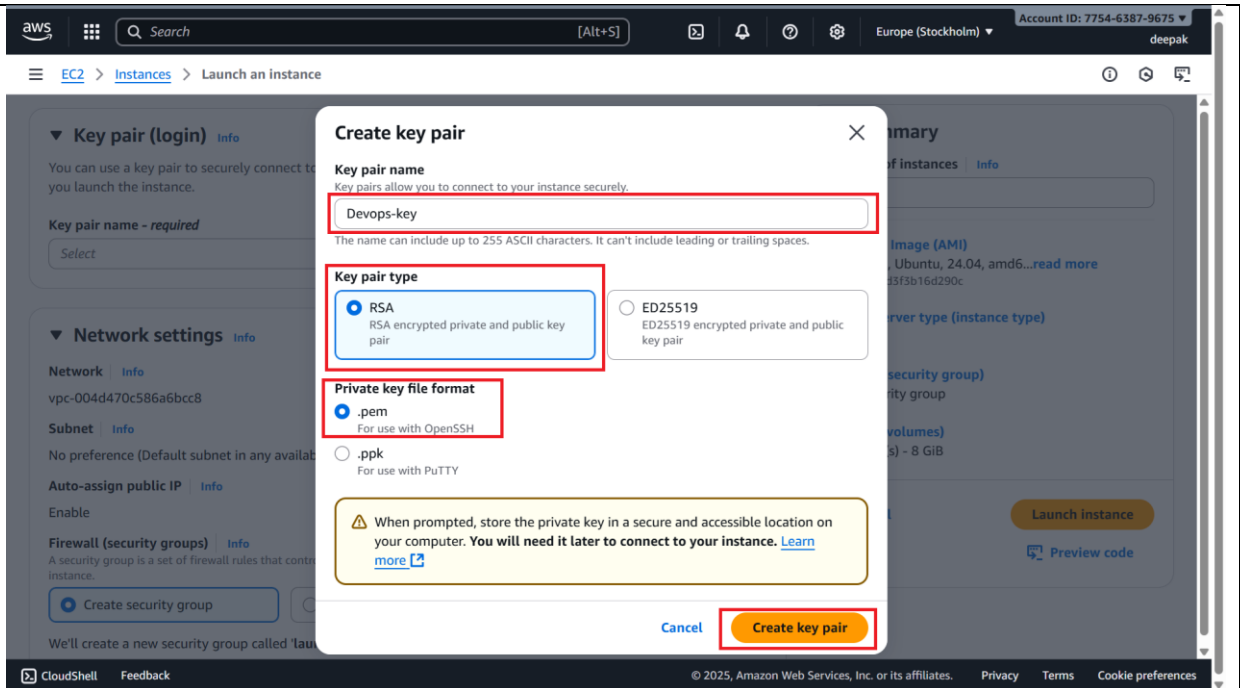
	
3	3. Click Launch Instance to begin setup 
4	3. Choose an Amazon Machine Image (AMI) • Select Ubuntu Server 22.04 LTS or Ubuntu 24.04 LTS (recommended for longer support) • Ensure it is marked Free Tier eligible

5 4. Select Instance Type

- Choose **t2.micro** (1 vCPU, 1 GB RAM) – Free Tier eligible

6 Configure Key Pair

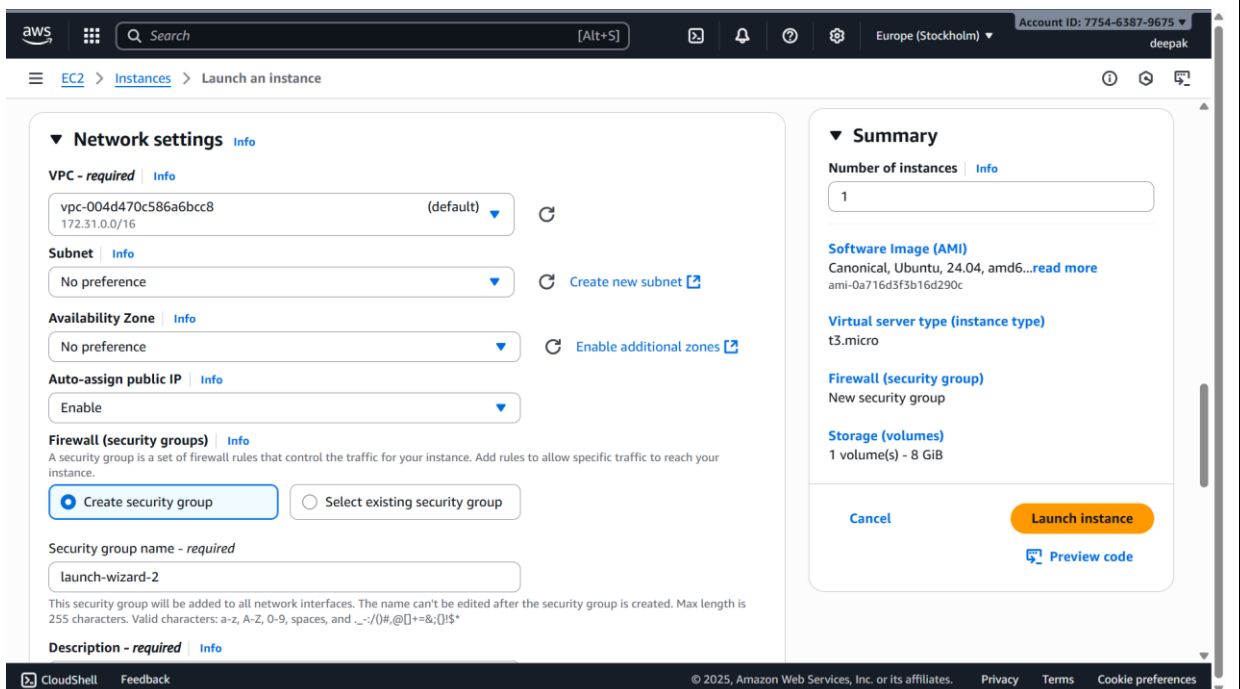
- Create a new key pair or use an existing one
- Choose .pem format for Linux/macOS or .ppk for Windows
- Download and store securely (you won't be able to download it again)

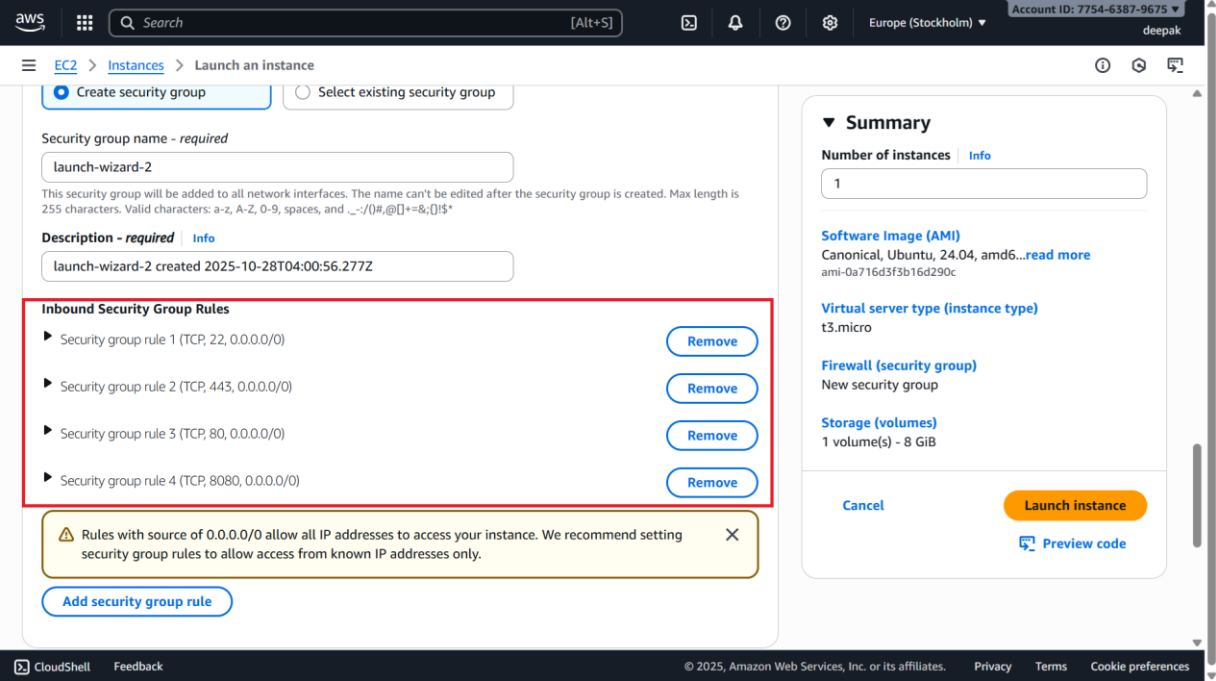
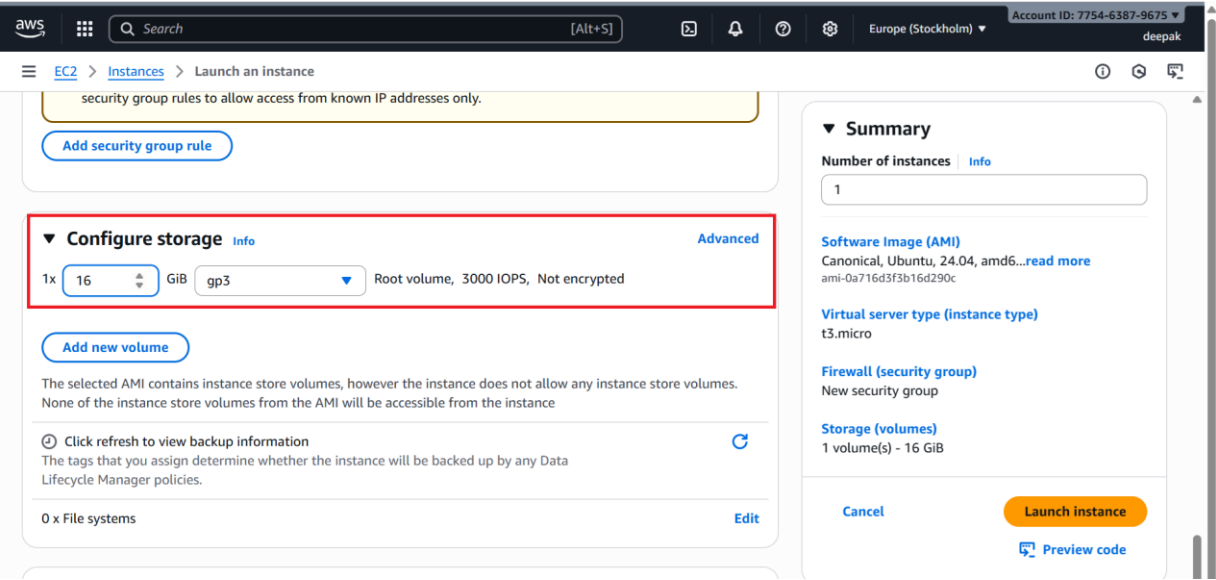


7

Configure Network Settings

- Use default VPC and subnet
- Create or select a **Security Group**
 - Allow **SSH (port 22)** from your IP
 - Optionally allow **HTTP (port 80)** and **HTTPS (port 443)** if hosting a web server



	
8	7. Configure Storage • Default is 8 GB SSD (gp3) • Free Tier allows up to 30 GB of gp3 storage 
9	Review and Launch • Review all settings • Click Launch Instance

aws Search [Alt+S] Europe (Stockholm) Account ID: 7754-6387-9675 deepak

EC2 > Instances > Launch an instance

security group rules to allow access from known IP addresses only.

[Add security group rule](#)

▼ **Configure storage** Info Advanced

1x 16 GiB gp3 Root volume, 3000 IOPS, Not encrypted

[Add new volume](#)

The selected AMI contains instance store volumes, however the instance does not allow any instance store volumes. None of the instance store volumes from the AMI will be accessible from the instance

Click refresh to view backup information
The tags that you assign determine whether the instance will be backed up by any Data Lifecycle Manager policies.

0 x File systems Edit

► **Advanced details** Info

▼ **Summary**

Number of instances Info
1

Software Image (AMI)
Canonical, Ubuntu, 24.04, amd64...[read more](#)
ami-0a716d3f3b16d290c

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 16 GiB

Cancel [Launch instance](#) [Preview code](#)

10 EC2 instance is created: Devops

aws Search [Alt+S] Europe (Stockholm) Account ID: 7754-6387-9675 deepak

EC2 > Instances

EC2

Dashboard
AWS Global View
Events

▼ **Instances**

Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New

▼ **Images**

AMIs
AMI Catalog

▼ **Elastic Block Store**

Volumes

Instances (1) Info Last updated less than a minute ago Connect Instance state Actions Launch instances

Find Instance by attribute or tag (case-sensitive)

Name = DevOps Clear filters

All states

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status
☐	DevOps	i-0bdba72360decb957	Running	t3.micro	Initializing	View alarms +

Select an instance

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

11

Connect to Your Instance

The screenshot shows the AWS Management Console 'Instances' page. The 'Connect' button is highlighted with a red box. Below the table, the details for instance 'i-0bdba72360decb957' are shown, including its public IP address and state.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status
DevOps	i-0bdba72360decb957	Running	t3.micro	Initializing	View alarms +

i-0bdba72360decb957 (DevOps)

Instance summary

- Instance ID: i-0bdba72360decb957
- Public IPv4 address: 16.171.177.177 | [open address](#)
- Private IPv4 addresses: 172.31.42.74
- Instance state: Running
- Public DNS: ec2-16-171-177-177.eu-north-1.compute.amazonaws.com | [open address](#)

Connect to Your Instance

The screenshot shows the AWS Management Console 'Connect to instance' page. The 'EC2 Instance Connect' tab is highlighted with a red box. The 'Connection type' section shows 'Connect using a Public IP' selected, with the public IP address 16.171.177.177. The 'Username' field is set to 'ubuntu'. The 'Connect' button is highlighted with a red box.

EC2 Instance Connect | Session Manager | SSH client | EC2 serial console

Instance ID: i-0bdba72360decb957 (DevOps)

Connection type

- ☒ Connect using a Public IP
Connect using a public IPv4 or IPv6 address
- ☐ Connect using a Private IP
Connect using a private IP address and a VPC endpoint

Public IPv4 address
16.171.177.177

Username
ubuntu

Note: In most cases, the default username, ubuntu, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

[Cancel](#) [Connect](#)

12

Update System and Install Java

- `sudo apt update`

```
aws Search [Alt+S] Europe (Stockholm) Account ID: 7754-6387-9675 deepak

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-42-74:~$ sudo apt update
Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:5 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:6 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:7 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:8 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:9 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:10 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:11 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:12 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1545 kB]
Get:13 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [294 kB]
Get:14 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:15 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [15.4 kB]
Get:16 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1498 kB]
```

i-0bdba72360decb957 (DevOps)

- `sudo apt install openjdk-17-jdk -y`

```
aws Search [Alt+S] Europe (Stockholm) Account ID: 7754-6387-9675 deepak

Reading state information... Done
55 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-42-74:~$ sudo apt install openjdk-17-jdk -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  adwaita-icon-theme alsa-topology-conf alsa-ucm-conf at-spi2-common at-spi2-core ca-certificates-java dconf-gsettings-backend
  dconf-service fontconfig fontconfig-config fonts-dejavu-core fonts-dejavu-extra fonts-dejavu-mono gsettings-desktop-schemas
  gtk-update-icon-cache hicolor-icon-theme humanity-icon-theme java-common libasound2-data libasound2t64 libatk-bridge2.0-0t64
  libatk-wrapper-java libatk-wrapper-java-jni libatk1.0-0t64 libatspi2.0-0t64 libavahi-client3 libavahi-common-data libavahi-common3
  libcairo-gobject2 libcairo2 libcups2t64 libdatrie1 libdconf1 libdeflate0 libdrm-amdgpu1 libdrm-intel1 libfontconfig1 libgail-common
  libgail18t64 libgbml libgdk-pixbuf-2.0-0 libgdk-pixbuf2.0-bin libgdk-pixbuf2.0-common libgif7 libgl1 libgl1-mesa-dri libglvnd0
  libglx-mesa0 libglx0 libgraphite2-3 libgtk2.0-0t64 libgtk2.0-bin libgtk2.0-common libharfbuzz0b libice-dev libice6 libjbig0
  libjpeg-turbo8 libjpeg8 liblcms2-2 liblerc4 libllvmlib2 libpango-1.0-0 libpangocairo-1.0-0 libpangoft2-1.0-0 libpciaccess0 libpcsc-lite1
  libpixmap-1-0 libpthread-stubs0-dev librsync2-2 librsync2-common libsharpyuv0 libsm-dev libsm6 libthai-data libthai0 libtiff6 libvulkan1
  libwayland-client0 libwayland-server0 libwebp7 libx11-dev libx11-xcb1 libxau-dev libxaw7 libxcb-dri3-0 libxcb-glx0 libxcb-present0
  libxcb-randr0 libxcb-render0 libxcb-shape0 libxcb-shm0 libxcb-sync1 libxcb-xfixes0 libxcb1-dev libxcomposite1 libxcursor1 libxdamage1
  libxdmcp-dev libxfixes3 libxft2 libxi6 libxinerama1 libxkbfile1 libxmu6 libxpm4 libxrandr2 libxrender1 libxshmfence1 libxt-dev libxt6t64
  libxtest6 libxv1 libxxf86dgal libxxf86vml mesa-libgallium mesa-vulkan-drivers openjdk-17-jdk-headless openjdk-17-jre
  openjdk-17-jre-headless session-migration ubuntu-mono x11-common x11-utils x11proto-dev xorg-sgml-doctools xtrans-dev
Suggested packages:
  default-jre alsa-utils libasound2-plugins cups-common gvfs libice-doc liblcms2-utils pcscd librsync2-bin libsm-doc libx11-doc libxcb-doc
  libxt-doc openjdk-17-demo openjdk-17-source visualvm libnss-mdns fonts-ipafont-gothic fonts-ipafont-mincho fonts-wqy-microhei
  fonts-wqy-zenhei fonts-indic mesa-utils
Recommended packages:
  luit
The following NEW packages will be installed:
```

i-0bdba72360decb957 (DevOps)

14 **Install sonarqube using following command:**

Install docker using command: `sudo apt install docker.io`

Add current user in docker group using command: `sudo usermod -aG docker $USER`

Reboot machine using command: `sudo reboot`

Note: Docker group is already created as part of docker installation.

Check using command: `sudo cat /etc/group`.

If docker group is not present create using command: `sudo newgrp docker`

```
devopsacc2025@devops:~$ sudo apt install docker.io
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base libidn11 pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools cgroupfs-mount | cgroup-lite debotstrap docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils
The following NEW packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base docker.io libidn11 pigz runc ubuntu-fan
0 upgraded, 9 newly installed, 0 to remove and 8 not upgraded.
Need to get 71.6 MB of archives.
After this operation, 301 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://us-central1.gce.archive.ubuntu.com/ubuntu focal/universe amd64 pigz amd64 2.4-1 [57.4 kB]
Get:2 http://us-central1.gce.archive.ubuntu.com/ubuntu focal/main amd64 bridge-utils amd64 1.6-2ubuntu1 [30.5 kB]
Get:3 http://us-central1.gce.archive.ubuntu.com/ubuntu focal-updates/main amd64 runc amd64 1.1.12-0ubuntu2-20.04.1 [8066 kb]
Get:4 http://us-central1.gce.archive.ubuntu.com/ubuntu focal-updates/main amd64 containerd amd64 1.7.24-0ubuntu1-20.04.1 [33.6 MB]
Get:5 http://us-central1.gce.archive.ubuntu.com/ubuntu focal-updates/main amd64 dns-root-data all 2024071801-ubuntu0.20.04.1 [6128 B]
Get:6 http://us-central1.gce.archive.ubuntu.com/ubuntu focal/main amd64 libidn11 amd64 1.33-2.2ubuntu2 [46.2 kB]
Get:7 http://us-central1.gce.archive.ubuntu.com/ubuntu focal-updates/main amd64 dnsmasq-base amd64 2.90-0ubuntu0.20.04.1 [350 kB]
Get:8 http://us-central1.gce.archive.ubuntu.com/ubuntu focal-updates/universe amd64 docker.io amd64 26.1.3-0ubuntu1-20.04.1 [29.5 MB]
Get:9 http://us-central1.gce.archive.ubuntu.com/ubuntu focal-updates/main amd64 ubuntu-fan all 0.12.13ubuntu0.1 [34.4 kB]
Fetched 71.6 MB in 1s (48.3 MB/s)
```

If prompted press Y

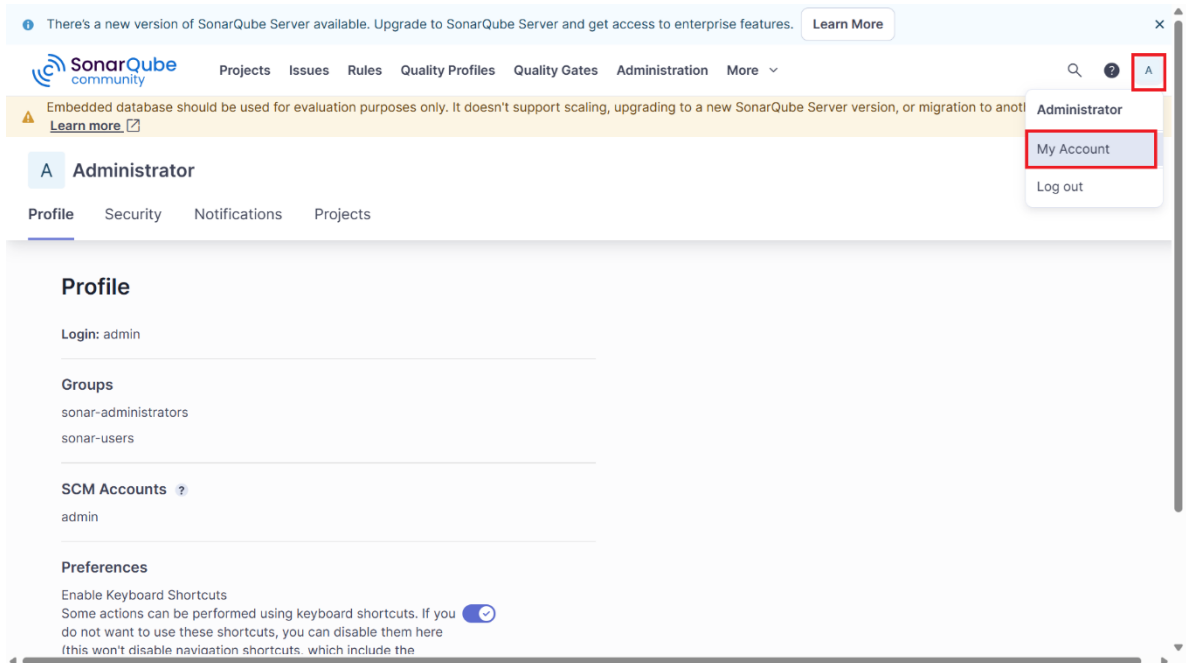
`docker run --rm -d -p 9000:9000 -v sonarqube_extensions:/opt/sonarqube/extensions sonarqube`

run on browser `http://<External ip>:9000/`

15 Create SoneQube Token for Static Code Analysis

navigate to your user profile on the SonarQube platform.

Go to "My Account"



16 Select : Security

Specify token name in generate Token

Select Generate

New token is generated. Copy it for further use.

There's a new version of SonarQube Server available. Upgrade to SonarQube Server and get access to enterprise features.

Learn More

X

SonarQube

community

ProjectsIssuesRulesQuality ProfilesQuality GatesAdministrationMore

SearchHelpAdmin

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine.

Learn more

Administrator

ProfileSecurityNotificationsProjects

If you want to enhance security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services, you can provide a User Token as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password going through your network.

Generate Tokens

Name

Type

Expires in

Generate

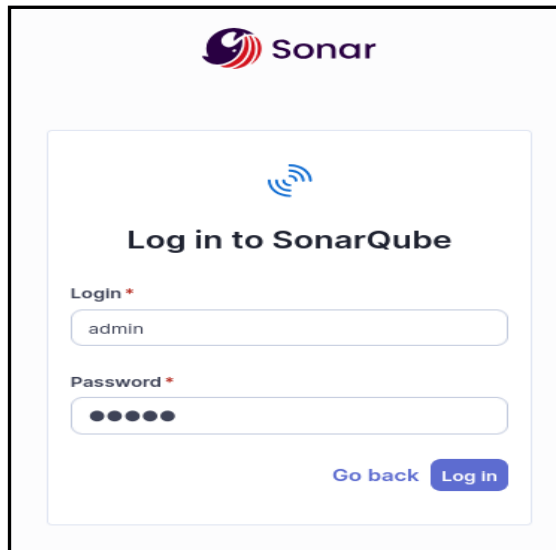
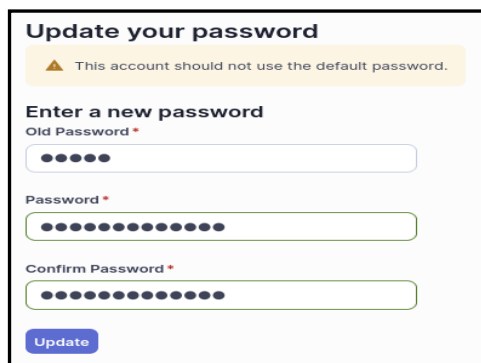
DevOps

Global Analysis Token

30 days

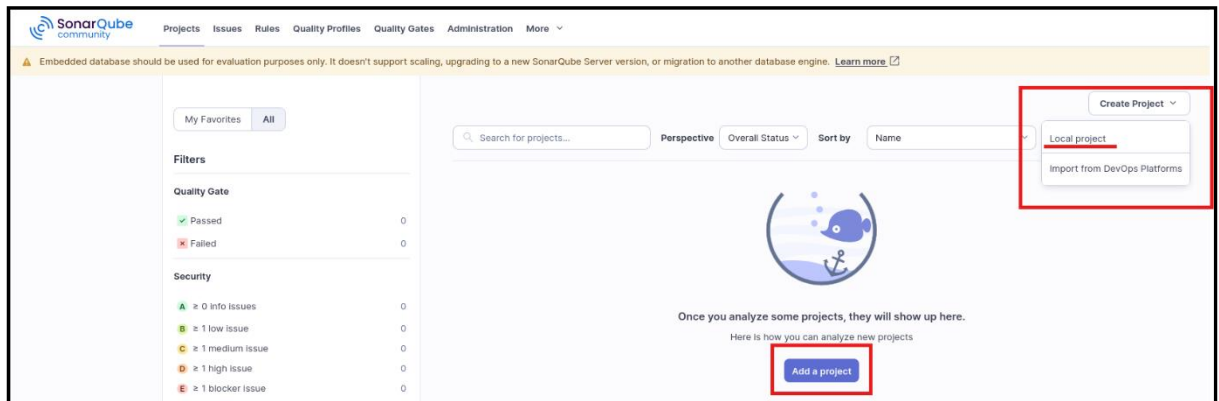
Name	Type	Project	Last use	Created	Expiration
No tokens					

Activity 2: Creating Project in SonarQube

1	Open terminal and run below command to start SonarQube server: <pre><i>docker run --rm -d --name sonarqube-server -p 9000:9000 -v sonarqube_extensions:/opt/sonarqube/extensions sonarqube</i></pre>
2	Open browser and hit http://localhost:9000 Enter username and password as admin The image shows the SonarQube login interface. At the top is the Sonar logo. Below it is a blue Sonar icon. The heading "Log in to SonarQube" is centered. There are two input fields: "Login *" with the text "admin" and "Password *" with masked dots. At the bottom right are "Go back" and "Log In" buttons.
3	After logging change the password when prompted. The image shows the "Update your password" page. It has a yellow warning banner that says "This account should not use the default password." Below is the heading "Enter a new password". There are three input fields: "Old Password *" with masked dots, "Password *" with masked dots, and "Confirm Password *" with masked dots. At the bottom is an "Update" button.

4

Create a local project

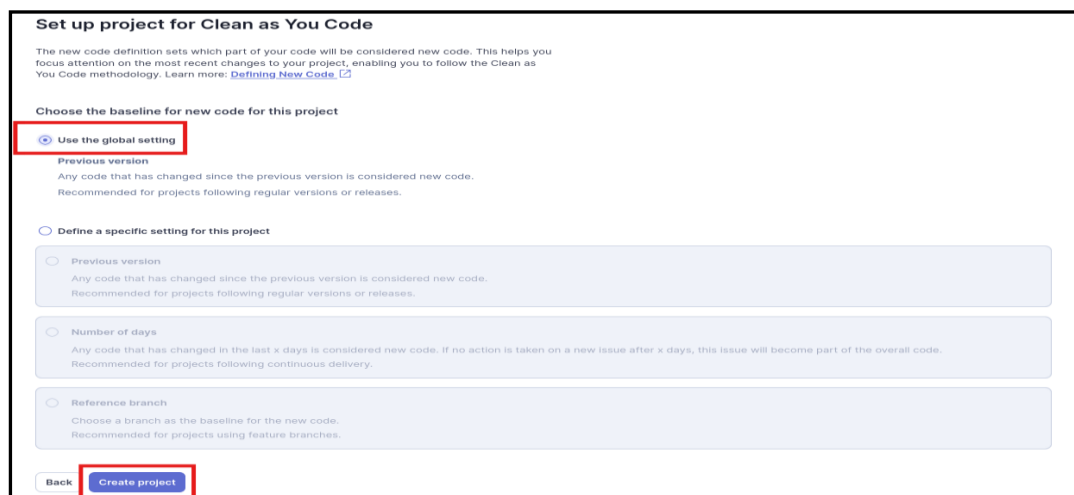


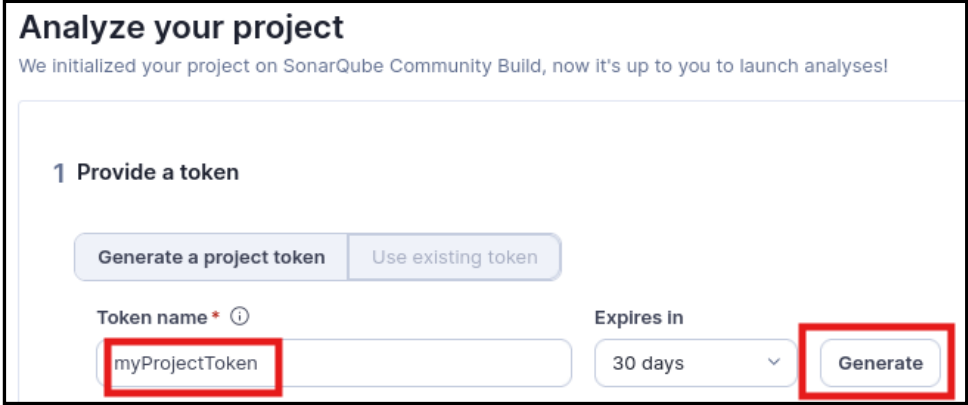
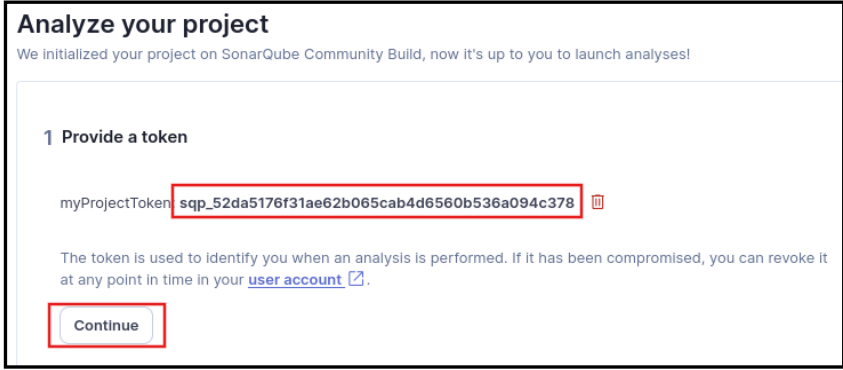
Provide a **suitable name** and a **unique key** for the project and click on **next** button.

These values will be required later when configuring the **SonarQube integration** in the Spring Boot project.

In the **“Set up project for Clean as You Code”** screen:

- Choose **“Use the global setting”** to apply the default configuration, where any code changed since the previous version is considered *new code*. This option is recommended for projects following regular releases or versioning practices.
- Click **“Create project”** to proceed.



5	In the “Analysis Method” setup screen, Click on “Locally” under the available analysis options. This option allows you to run SonarQube analysis directly from your local system using the Sonar Scanner or Maven command. Provide a suitable token name that identifies this project’s analysis purpose. This token will be used for authenticating and analyzing the application’s static code. Once the name is entered, click on the “Generate” button to create the token. 
6	After the token is generated, copy the token value and store it securely. This token will be required in subsequent steps for SonarQube configuration and project analysis execution and then click on continue button. 
7	In the “Run analysis on your project” section, select the appropriate build tool for your application. For Maven-based projects, choose the “Maven” option and execute the following command in the project’s root directory terminal: <pre>mvn clean verify sonar:sonar \ -Dsonar.projectKey=<<Project_key>> \ -Dsonar.projectName=<<Sonar_Project-Name>> \ -Dsonar.host.url=http://localhost:9000 \ -Dsonar.token=<<Sonar-Token>></pre> Note: 1. Replace <<Project_key>> with the project key created in SonarQube. 2. Replace <<Sonar_Project-Name>> with the project name created in SonarQube. 3. Replace <<Sonar-Token>> with the token which is created by SonarQube Server.

Activity 3: Static Code Analysis using SonarQube

1	Download all demos. Import the spring boot project <i>Spring_Boot_Project_Structure.zip</i> into the eclipse workspace. Note: If spring boot project does not exist then create a new spring boot project.
2	Add below to the pom.xml under <properties> tag: <pre><sonar.projectKey> <<SonarQube Project key>> </sonar.projectKey> <sonar.projectName> << SonarQube Project name >> </sonar.projectName></pre> Note: Replace <<SonarQube Project key>> and << SonarQube Project name >> with the respective project key and project name that is created in the SonarQube. Example: If the project key is <i>myProject</i> and project name is <i>myProject</i> then add below lines under <properties> tag in pom.xml: <pre><sonar.projectKey>myProject</sonar.projectKey> <sonar.projectName>myProject</sonar.projectName></pre>
3	Open terminal and navigate to the project root location using cd command. Run maven goal <i>mvn clean install</i> in the terminal. Run the following SonarQube command in the terminal to perform static code analysis and review the quality results: <pre>mvn sonar:sonar -Dsonar.host.url=http://localhost:9000 -Dsonar.login=<<sonar_token>></pre> Note: Replace <<sonar_token>> with the valid authentication token generated from your SonarQube server. Example: If the sonar token is <i>sqp_52da5176f31ae62b065cab4d6560b536a094c378</i> Then the command is as below: <pre>mvn sonar:sonar -Dsonar.host.url=http://localhost:9000 -Dsonar.login= sqp_52da5176f31ae62b065cab4d6560b536a094c378</pre>

4

Open the SonarQube UI in the browser and view the analysis

Quality Gate **Failed**

The last analysis has warnings. [See details](#)

New Code **2 failed** Overall Code

New Code: Since 1.0 Started 2 hours ago

2 conditions failed

3 Issues

0.0% Coverage
Is less than 80.0%

Fix issues before they fail your Quality Gate with SonarQube for IDE . Power up with

New Issues **FAILED**
3
Required = 0

Accepted Issues
0
Valid issues that were not fixed

Coverage
FAILED
0.0%

Duplications
0.0%
Required ≤ 2.0%

5

Click on the issues and check

My Issues **All** [Clear All Filters](#)

Looking for Bugs, Vulnerabilities, or Code Smells? If your team prefers working with these types, change it in the [settings](#)

Issues in new code

▼ **Software Quality**

Security 0

Reliability 0

Maintainability 3

▼ **Severity**

Blocker 0

High 0

Medium 3

Low 0

Info 0

☐ Bulk Change

Select issues Navigate to issue **3 Issues** **30min effort**

src/_/accenture/htt/web/controller/EmployeeController.java

☐ [Replace this use of System.out by a logger.](#) **Maintainability** **bad-practice** **cert** **Adaptability**

☐ Open ☐ Not assigned L30 · 10min effort · 2 hours ago

☐ [Replace this use of System.out by a logger.](#) **Maintainability** **bad-practice** **cert** **Adaptability**

☐ Open ☐ Not assigned L39 · 10min effort · 2 hours ago

☐ [Replace this use of System.out by a logger.](#) **Maintainability** **bad-practice** **cert** **Adaptability**

☐ Open ☐ Not assigned L55 · 10min effort · 2 hours ago

3 of 3 shown

6

Click on the analysis issue

```

25 EmployeeDAO employeeDAO;
26
27 @GetMapping(value = "emp/controller/getDetails", produces = MediaType.APPLICATION_JSON_VALUE)
28 public ResponseEntity<List<Employee>> getEmployeeDetails() {
29
30     System.out.println("getEmployeeDetails()");
31
32     List<Employee> listEmployee = new ArrayList<Employee>(employeeDAO.getAllEmployee());
33
34     return new ResponseEntity<List<Employee>>(listEmployee, HttpStatus.OK);
35 }
36
37 @GetMapping(value = "emp/controller/getDetailsById/{id}", produces = MediaType.APPLICATION_JSON_VALUE)
38 public ResponseEntity<Employee> getEmployeeDetailByEmployeeId(@PathVariable("id") int myId) {
39     Employee employee = employeeDAO.getEmployeeDetailsById(myId);
40     System.out.println("getEmployeeDetailByEmployeeId()");
41 }

```

Replace this use of System.out by a logger.

Replace this use of System.out by a logger.

3 of 3 shown

7

Change the respective code in eclipse by adding loggers.

Repeat **step 3** in this **activity 3** and observe the report in the SonarQube UI

Quality Gate ⓘ

 **Passed**

Last analysis 28 seconds ago

 The last analysis has warnings. [See details](#)

New Code

Overall Code

New Code: Since 1.0 Started 2 hours ago

New Issues

0

Required = 0

Accepted Issues

0

Valid issues that were not fixed

Coverage

Not computed

Duplications

0.0%

Required ≤ 3.0%

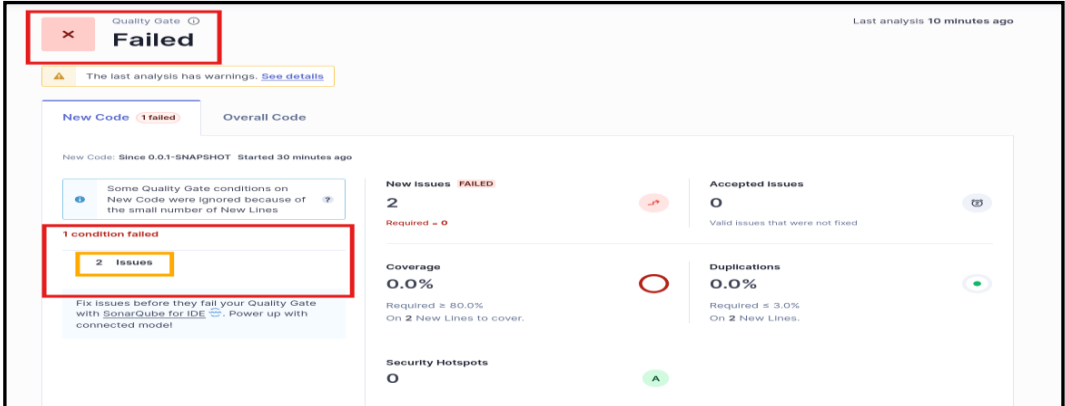
On 59 New Lines.

Security Hotspots

0

A

Activity 4: Static Code Analysis using Sonar Scanner

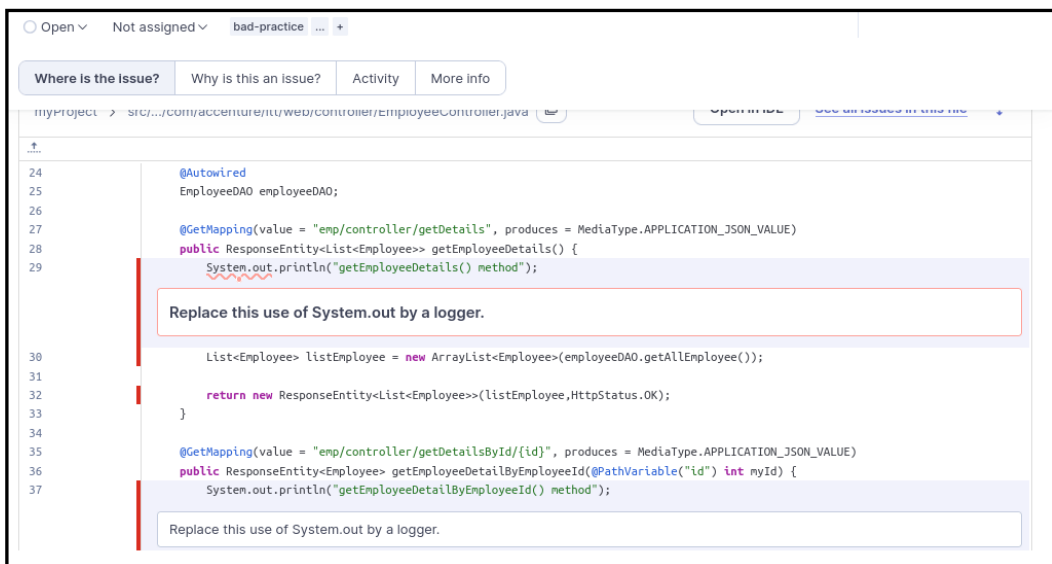
1	Import the spring boot project <i>Spring_Boot_Project_Sonar_Scanner.zip</i> into the eclipse workspace. Note: If spring boot project does not exist then create a new spring boot project. In the project's root directory, create a new file with below name sonar-project.properties and add the following configuration lines: <pre>sonar.projectKey=<<Project-Key>> sonar.projectName=<<Project-Name>> sonar.projectVersion=1.0 sonar.sources=src/main/java sonar.java.binaries=target/classes sonar.exclusions=**/test/** sonar.sourceEncoding=UTF-8 sonar.host.url=http://localhost:9000 sonar.login=<<sonar_token>></pre> Note: . 1. Replace <<Project_Key>> with the project key created in SonarQube. 2. Replace <<Project-Name>> with the project name created in SonarQube. 3. Replace <<sonar_token>> with the valid authentication token generated from your SonarQube server.
2	Open terminal and navigate to the project root location using cd command. Run maven goal <i>mvn clean install</i> in the terminal. Run the following SonarQube scanner command in the terminal to perform static code analysis and review the quality results: <i>sonar-scanner</i>
3	Open the SonarQube UI in the browser and view the analysis. 

Click on the issues and check



The screenshot shows a list of two SonarQube issues. Both issues are titled "Replace this use of System.out by a logger." and are categorized under "Maintainability". The first issue is labeled "L29" and the second "L37", both with an effort of "10min" and reported "21 minutes ago". The issues are currently in the "Open" state and are not assigned. A red box highlights the first issue's details.

Click on the analysis issue

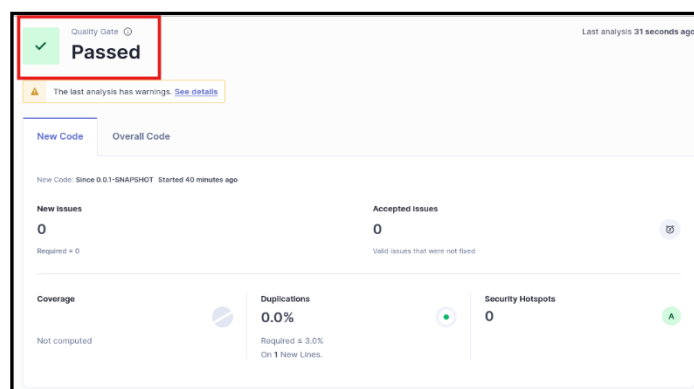


The screenshot displays the details of a SonarQube issue. The issue title is "Replace this use of System.out by a logger." and it is located at line 29 of the file "src/.../com/accnture/it/web/controller/EmployeeController.java". The issue is highlighted in the code editor, showing the following code snippet:

```
24 @Autowired
25 EmployeeDAO employeeDAO;
26
27 @GetMapping(value = "emp/controller/getDetails", produces = MediaType.APPLICATION_JSON_VALUE)
28 public ResponseEntity<List<Employee>> getEmployeeDetails() {
29     System.out.println("getEmployeeDetails() method");
30
31     List<Employee> listEmployee = new ArrayList<Employee>(employeeDAO.getAllEmployee());
32
33     return new ResponseEntity<List<Employee>>(listEmployee, HttpStatus.OK);
34 }
35
36 @GetMapping(value = "emp/controller/getDetailsById/{id}", produces = MediaType.APPLICATION_JSON_VALUE)
37 public ResponseEntity<Employee> getEmployeeDetailByEmployeeId(@PathVariable("id") int myId) {
38     System.out.println("getEmployeeDetailByEmployeeId() method");
39 }
```

Change the respective code in eclipse by adding loggers.

Repeat **step 2** in this **activity 4** and observe the report in the SonarQube UI



The screenshot shows the SonarQube dashboard. At the top, a green checkmark indicates the "Quality Gate" is "Passed". Below this, a warning icon indicates "The last analysis has warnings. See details". The dashboard displays the following metrics:

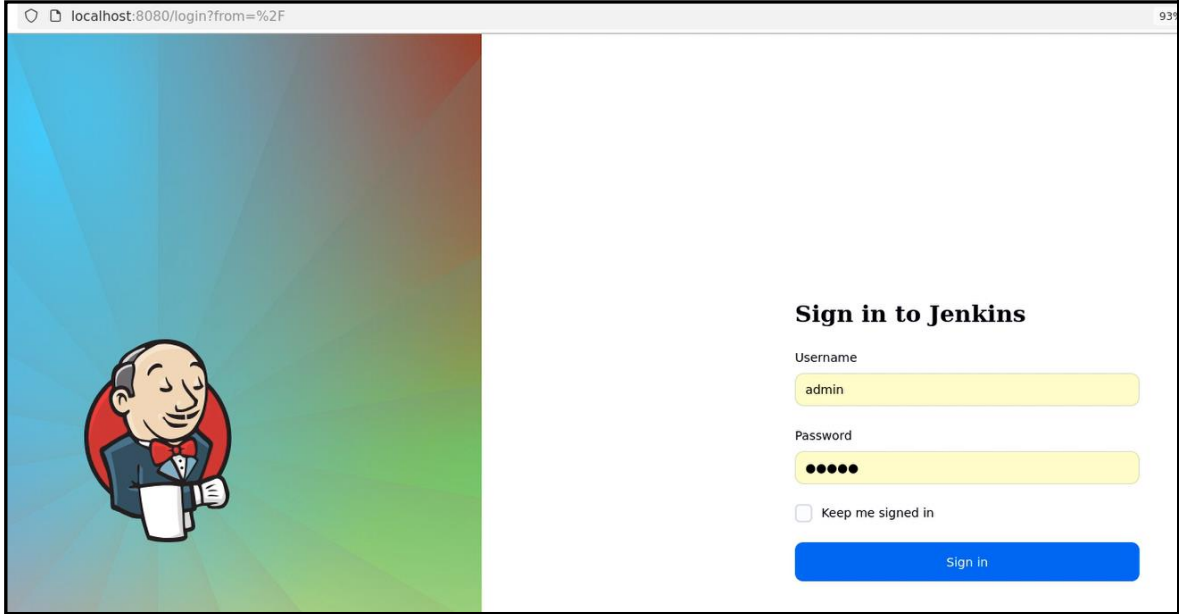
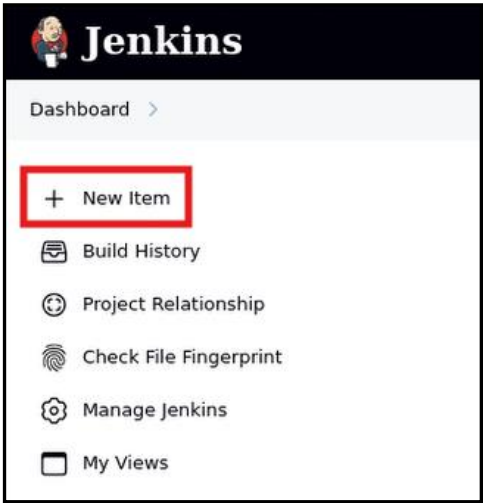
- New Issues:** 0 (Required: 0)
- Accepted Issues:** 0 (Valid issues that were not fixed)
- Coverage:** Not computed
- Duplications:** 0.0% (Required: ≤ 3.0%, On 1 New Lines)
- Security Hotspots:** 0 (Grade: A)

Activity 5: Static Code Analysis on spring boot application cloned from Gitlab

1	Clone the project repository using the provided GitLab URL to obtain a local copy of the source code.
2	Ensure that appropriate test cases are defined within the project to validate the application functionality and maintain code quality.
3	Navigate to the cloned repository's directory and open it in the terminal to perform subsequent build or analysis commands.
4	Check for static code analysis. Option 1: Using SonarQube - Run maven command <i>mvn clean install</i> in the terminal. - Run the below command in the terminal to perform code quality and coverage analysis using SonarQube: <i>mvn sonar:sonar -Dsonar.host.url=http://localhost:9000 -Dsonar.login=<<sonar token>></i> Note: Replace <<sonar token>> with the token generated by SonarQube Example: If the sonar token is Then the command is as below: <i>mvn sonar:sonar -Dsonar.host.url=http://localhost:9000 -Dsonar.login=</i> - Open the SonarQube user interface in a web browser and review the project analysis results to verify code quality, test coverage, and any reported issues. - If the analysis reports any failures or issues, review the findings and apply the necessary code fixes or configuration corrections to ensure compliance with project quality standards. - After implementing the fixes, commit the updated code to the local repository and push the changes to the GitLab origin branch - Repeat steps a, b, and c to re-run the build, test, and analysis processes until the project passes all quality checks successfully.

5	Option 2: Using Sonar Scanner a. In the project's root directory, create a new file with name sonar-project.properties and add the following configuration lines: <pre>sonar.projectKey=<<Project_Key>> sonar.projectName=<<Project_Name>> sonar.projectVersion=1.0 sonar.sources=src/main/java sonar.java.binaries=target/classes sonar.exclusions=**/test/** sonar.sourceEncoding=UTF-8 sonar.host.url=http://localhost:9000 sonar.login=<<sonar token>></pre> Note: 1. Replace <<Project_Key>> with the project key created in SonarQube. 2. Replace <<Sonar_Project-Name>> with the project name created in SonarQube. 3. Replace <<sonar token>> with the token generated by SonarQube. b. Run maven goal <i>mvn clean install</i> in the terminal. c. Run the below sonar scanner command in the terminal to check the analyzed static code: Sonar-scanner d. Open the SonarQube user interface in a web browser and review the project analysis results to verify code quality, test coverage, and any reported issues. e. If the analysis reports any failures or issues, review the findings and apply the necessary code fixes or configuration corrections to ensure compliance with project quality standards. f. After implementing the fixes, commit the updated code to the local repository and push the changes to the GitLab origin branch g. Repeat steps b, c and d to re-run the build, test, and analysis processes until the project passes all quality checks successfully.
---	--

Activity 6: Integrating SonarQube and Gitlab with Jenkins to build pipeline

1	Open a web browser and navigate to the following URL to access the Jenkins dashboard: http://localhost:8080  Note: Username = admin and password = admin
2	In the Jenkins dashboard, click on “New Item” to create a new project or pipeline. 

- 3 Provide a suitable **name** for the item, select the appropriate **project type** as ***Pipeline***, and click **OK** to proceed with configuration.

New Item

Enter an item name

StaticCodeAnalysisInPipeline

Select an item type

**Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

**Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.

**Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

**Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

**Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

**Multibranch Pipeline**
Creates a set of Pipeline projects according to detected branches in one SCM repository.

**Organization Folder**
Creates a set of multibranch project subfolders by scanning for repositories.

OK

- 4 Provide an appropriate **description** for the newly created Jenkins item to indicate its purpose or functionality.

General

Enabled ☒

Description

This is a demo to know how to implement the static code analysis in pipeline

Plain text [Preview](#)

- 5 Scroll down to the **Pipeline** section and set the **Definition** field to “**Pipeline script**”.

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3   stages {
4     stage('checkout source code') {
5       steps {
6         git branch: 'main', url: 'https://gitlab.com/myaccession-group/demo10013_spring_boot_project'
7       }
8     }
9     stage('static code analysis'){
10      steps{
11        withMaven(maven: 'maven3.9.9') {
12          sh 'mvn sonar:sonar -Dsonar.host.url=http://localhost:9000/ -Dsonar.login=sqa_9a05e444a32ab6128a273c9be49a2ea50dc7120'
13        }
14      }
15    }
16  }
17 }
```

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

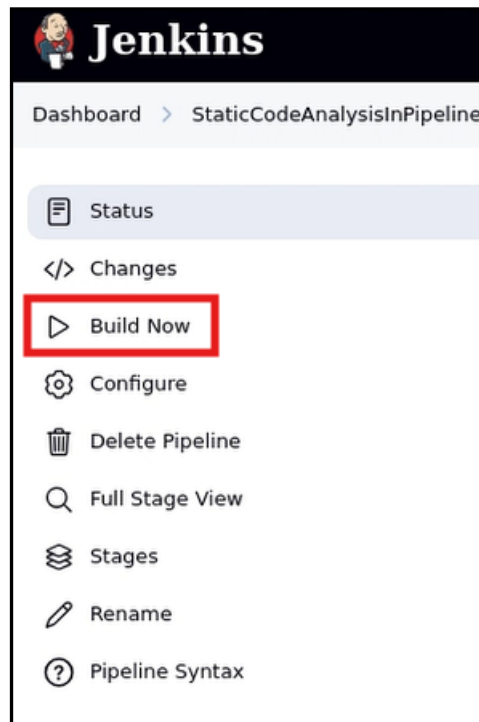
- 6 In the text area that appears, paste the following pipeline script and replace the placeholders with your actual GitLab repository URL and SonarQube token before saving the configuration.:

```
pipeline {
  agent any
  stages {
    stage('checkout source code') {
      steps {
        git branch: 'main', url:
        https://gitlab.com/<<your-group>>/<<your-project>>
      }
    }
    stage('static code analysis'){
      steps{
        withMaven(maven: 'maven3.9.9') {
          sh 'mvn sonar:sonar -
          Dsonar.host.url=http://localhost:9000/ -
          Dsonar.login=<<SonarQube token>> '
        }
      }
    }
  }
}
```

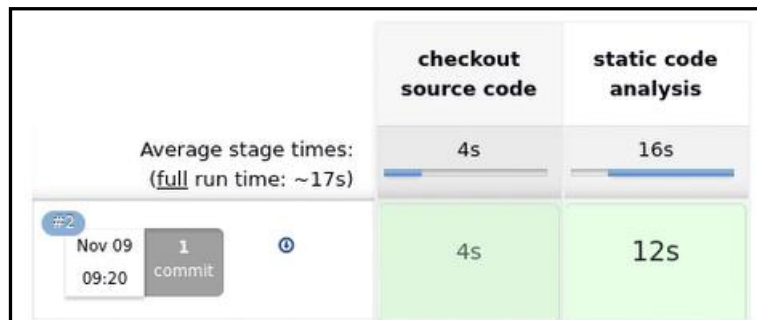
Note:

1. Replace <<your-group>>/<<your-project>> with your actual GitLab repository path.
2. Replace <<SonarQube token>> with a valid authentication token generated from your SonarQube instance.

- 7 Click on save button.
Click on **Build now** option



- 8 Check the build result.



- 9 Check the SonarQube UI for the analysis result.

If the analysis reports any failures or issues, review the findings and apply the necessary code fixes or configuration corrections to ensure compliance with project quality standards.

After implementing the fixes, commit the updated code to the local repository and push the changes to the GitLab origin branch